

LLM Agents Can See Code Repositories

Dongjian Ma*
2510332169@qq.com
Shanghai Jiao Tong University
Shanghai, China

Silin Chen*
cslsolow@gmail.com
Shanghai Jiao Tong University
Shanghai, China

Yufei Yang
qfrfyflc@stu.xjtu.edu.cn
Xi'an Jiaotong University
Xi'an, China

Yuling Shi
yuling.shi@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Yanfu Yan
yanfu@zju.edu.cn
Zhejiang University
Hangzhou, China

Xiaodong Gu†
xiaodong.gu@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Abstract

Coding agents powered by large language models (LLMs) have demonstrated remarkable proficiency in software engineering tasks. Yet modern coding agents rely almost entirely on text, leaving a major gap between how human developers and agents comprehend software repositories. Human developers actually "see" code repositories, where folder hierarchies, file dependencies, and syntax highlighting all convey critical semantics. With the rapid progress of Multimodal Large Language Models (MLLMs), an open question is whether these additional modalities can help LLMs understand repositories more effectively or efficiently. In this paper, we conduct the first systematic empirical study on multimodal foundation models for repository-level tasks. Our experiments across four modern multimodal models reveal that while a vision-only context representation degrades performance and inflates token costs, integrating visualized context graphs as a supplementary modality can help agents grasp the repository more efficiently. Specifically, providing agents with visual structural context alongside standard text interfaces reduces input token consumption by up to 26% while maintaining or improving issue-resolution accuracy. Furthermore, we demonstrate that visual tools are most effective when utilized during the fault localization stage and when agents autonomously dictate their exploration depth. Our findings highlight a promising hybrid-modality pathway for the design of next-generation coding agents. Our code and data are available at <https://github.com/cslsolow/SeeRepo>.

1 Introduction

Coding agents empowered by large language models (LLMs) have shown remarkable capabilities in software engineering tasks, including resolving issues in large repositories [12, 35]. Most existing agents interact with repositories through tokenized text: source code, documentation, and execution feedback are flattened into sequences for reasoning and planning. While this text-centric paradigm has driven rapid progress, it raises an open question: *Is text the most effective modality for presenting repository context to modern foundation models?*

To developers, code repositories comprise artifacts from multiple modalities, including not only textual source code and documentation but also structural relationships such as function dependencies, etc.. However, existing techniques for repository understanding

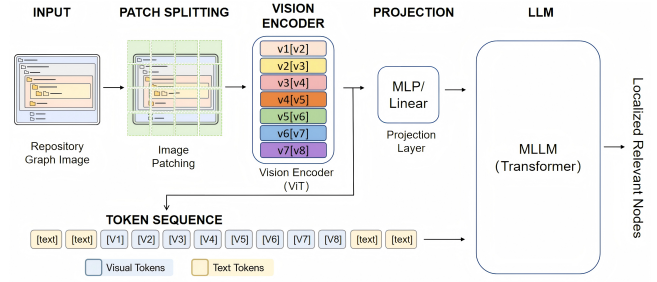


Figure 1: Process of how MLLMs perceive multimodal rendering of a code repository. At each agent step, the repository is provided as both image and text: the image is split into patches, encoded by a ViT into visual tokens, projected into the LLM embedding space, and concatenated with text tokens, preserving spatial topology for dependency reasoning.

across a range of SE tasks typically rely on linearizing these heterogeneous artifacts into sequential inputs. As a result, models are required to infer structural information that was originally conveyed through non-linear or visual representations. Recovering such organizations and relationships can be challenging under limited context budgets. While prior work has explored structural representations of code repositories, such as graph-based abstractions [6, 11, 16], the information ultimately consumed by models at inference time remains predominantly in the form of text tokens. Even when graph encodings are used, they are typically linearized for model input [15], which may lead to the loss of important relational cues. In contrast, visual representations of repositories can expose additional signals—such as two-dimensional layout and stable spatial grouping—that are not naturally captured by linear text. More broadly, visual context may provide richer information per unit of prompt, potentially improving an agent’s ability to remain oriented, retrieve relevant context, and perform accurate edits in long-horizon repository workflows.

In this paper, we conduct the first empirical study on multimodal foundation models for repository-level tasks. We study the effects of shifting repository representations toward visual modalities. Specifically, we evaluate four multimodal models—GPT-5-mini [20], GPT-5.1 [21], Doubao-Seed-2.0-Lite [3], and Kimi K2.5 [17]—and analyze how design choices—including representation modality, image-text balance, visualization layout design, and visualization

*Dongjian Ma and Silin Chen contributed equally to this work.

†Corresponding author.

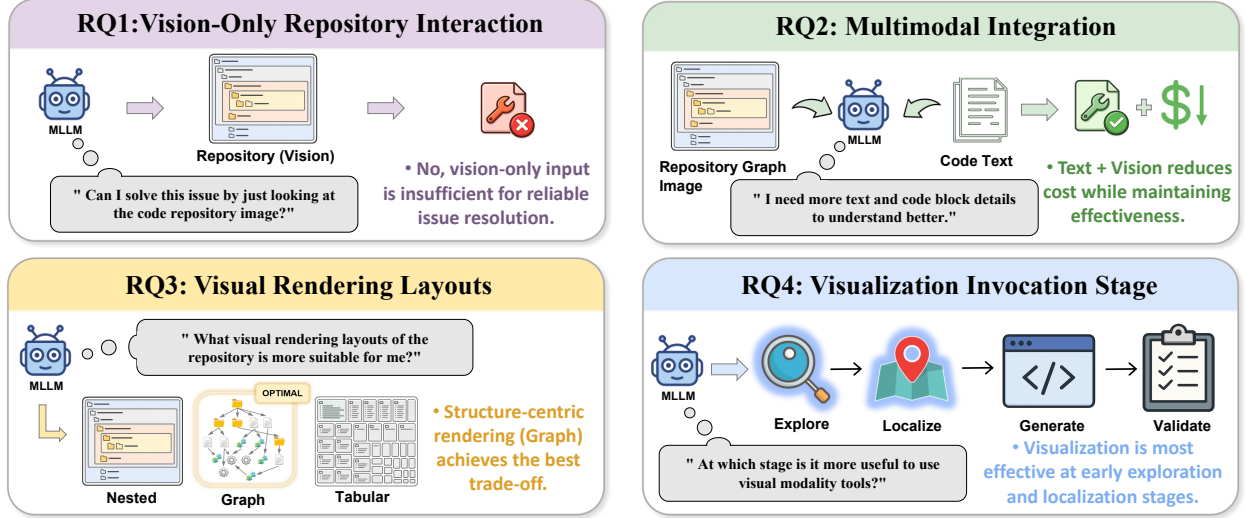


Figure 2: Overview of the Study Design and Core Findings

tool invocation stages—affect agent performance and efficiency. To support this study, we build SeeRepo, a multimodal augmentation for coding agents on repository-level issue resolution. It integrates visual graph renderings of repository structure with standard text-based code access and editing, enabling agents to combine structural awareness from vision with symbolic precision from text. Concretely, SeeRepo uses AST-based static analysis to construct multi-relation dependency graphs capturing containment, import, invocation, and inheritance relationships. Given a query node, it renders a Graphviz subgraph centered on that node as a PNG image, which the agent receives alongside conventional text-based code access. This hybrid interface allows the agent to leverage spatial structure from vision while retaining symbolic precision from text. Our study is guided by four research questions:

RQ1: How effective are current MLLMs at repository-level issue resolution? We investigate whether visual representations are effective in repository-level understanding. We set the Mini-SWE-Agent in a vision-only mode, where bash navigation commands return graph images instead of texts. We evaluate three representative MLLMs and compare the performance of vision-only modality against traditional text modality.

Findings. Vision-only interaction significantly degrades resolution accuracy across all three models: GPT-5-mini drops from 55.0% to 41.4% (−13.6), Doubao-Seed-2.0-Lite drops sharply from 51.0% to 16.9% (−34.1), and Kimi K2.5 drops from 70.3% to 55.0% (−15.3). Contrary to expectations, token cost surges rather than decreases—GPT-5-mini incurs 42% higher cost, Doubao exhibits 268% cost inflation, and Kimi K2.5 sees a 27% increase. Agents deprived of text access resort to repeated graph queries to compensate for missing symbolic information, accumulating high overhead without recovering accuracy. This suggests that current MLLMs are heavily reliant on symbolic cues for efficient code reasoning [33], and raw visual graphs alone cannot provide sufficient semantic guidance.

RQ2: How to integrate MLLMs into current agentic frameworks for repository-level issue resolution? Having observed that vision-only repository interaction degrades the performance of issue resolution, we wonder whether a hybrid *text+vision* context can combine the strengths of both modalities and thus enhance the performance of coding agents. We integrate the vision representation of the code repository as a supplementary context alongside the standard bash-based interface, and compare the performance under various models.

Findings. Integrating SeeRepo as a supplementary modality significantly reduces the agent’s token cost while maintaining or improving resolution accuracy. On GPT-5-mini, Pass@1 improves to 55.4% (+0.4) with input tokens reduced by 25% and cost by 26%. GPT-5.1 achieves a 46% cost reduction despite a minor accuracy dip (−2.2). Kimi K2.5 simultaneously improves Pass@1 by 1.8 points (68.8%→70.6%) and reduces cost by 3%. Doubao-Seed-2.0-Lite gains +1.0 Pass@1 with a 6% cost reduction. The same trend also transfers to additional GPT-5-mini evaluations on SWE-Rebench Leaderboard 2026.03 [2] and SWE-QA [23], where multimodal context preserves or improves effectiveness while reducing overall interaction cost. The results suggest that multi-modal repository representation can help agents grasp code context more efficiently.

RQ3: How do different visual layouts affect MLLMs’ ability for repository issue resolution? Having established that multimodal representation helps coding agent understand repository more efficiently, we examine which visual rendering strategy yields the best efficiency. We experiment with three visual rendering strategies (graph, nested, and tabular) at different hierarchical depths (fixed vs. dynamic) and compare with the text-only baseline.

Findings. All three visual layouts outperform the text-only baseline in token efficiency. Graph-based layout achieves the greatest token reduction (−25% input tokens, −26% cost) with a Pass@1 of 55.4% (+0.4); nested and tabular layouts trade slightly higher token cost for marginal accuracy gains of +0.8 and +1.2 respectively. In

terms of hierarchical depths, the dynamic depth strategy achieves a competitive Pass@1 gain (+0.4) while delivering the largest reductions in input tokens (−25%) and cost (−26%) across all depth configurations.

RQ4: Which stage are visual tools most effective when utilized for software issue resolution? We integrate vision representations into one of the three stages in the issue resolution pipeline—localization, repair, and patch validation, respectively—and analyze how visualization affects performance in each stage.

Findings. Visualization is most effective when invoked at the fault localization stage: a multimodal agent equipped with SeeRepo achieves Pass@1 of 55.4% (+0.4) with around 25% reduction in token cost.

To sum up, this paper makes the following contributions:

- We present the first systematic, large-scale study of visual repository representations for coding agents on the software issue resolution task.
- We empirically demonstrate a performance boundary between vision-only and multimodal repository representations: while vision-only repository representation is ineffective for issue resolution, combining both textual and visual representations yields a substantially better trade-off between effectiveness and efficiency.
- We design and implement SeeRepo as an experimental framework for studying repository visualization in coding agents. Through extensive experiments, we further show that structure-centric renderings and early-stage visualization during exploration and localization are the most effective ways to leverage visual context.

2 Background

2.1 Multimodal Large Language Models

Multimodal Large Language Models (MLLMs) extend traditional language models by incorporating visual perception, transforming images into sequences of visual tokens that can be jointly processed with text via a unified Transformer architecture [8]. As illustrated in Figure 1, an input image $I \in \mathbb{R}^{H \times W \times 3}$ is divided into N fixed-size patches $\{p_i\}_{i=1}^N$, each encoded by a vision encoder (e.g., ViT) into a dense visual embedding $v_i \in \mathbb{R}^{d_v}$. A projection layer f_θ , typically learned, maps these embeddings into the language model’s token space:

$$\tilde{v}_i = f_\theta(v_i) \in \mathbb{R}^d, \quad (1)$$

producing a visual token sequence $[\tilde{v}_1, \dots, \tilde{v}_N]$ that is concatenated with text tokens and processed by the Transformer. The 2D spatial arrangement of patches is approximately preserved via positional embeddings, enabling cross-modal attention to align visual regions with textual semantics. Consequently, MLLMs can leverage visual layouts to capture global context and relational structure that are difficult to represent purely through linear text sequences, even in software engineering contexts [7, 37].

2.2 Visual Perception of Repository Structure

Software repositories naturally exhibit rich topological structures, including dependency graphs, call relations, and modular hierarchies, which encode global context about program organization.

Formally, a repository can be modeled as a directed heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R})$ [6], where nodes $v \in \mathcal{V}$ represent files, classes, and functions with type $\phi(v) \in \mathcal{A}$, and edges $(u, v, r) \in \mathcal{E}$ capture typed relationships $r \in \mathcal{R} = \{\text{contains, imports, inherits, invokes}\}$. Multiple edge types are allowed between node pairs.

Prior approaches predominantly serialize such structural information into text [11, 15, 16], which can obscure higher-order relationships and introduce substantial token overhead. Recent work has begun exploring visual modalities for software engineering tasks [10, 29], but a systematic study of visual repository representations remains absent. By rendering $\mathcal{G}$ as an image, spatial locality and connectivity patterns are preserved: node positions encode module membership, edge trajectories encode dependency direction, and cluster boundaries in the rendering typically correspond to architectural boundaries. Treating repository graphs as images allows MLLMs to leverage their visual reasoning capabilities to potentially perceive structural dependencies and global organization, enabling structure-aware understanding that complements textual code representations.

3 Experimental Setup

3.1 Benchmark and Metrics

We conduct our main experiments on **SWE-bench Verified** [19], a human-curated subset of SWE-bench [12] released by OpenAI to provide more reliable and reproducible evaluations of AI agents on real-world software engineering tasks. Unlike the original benchmark, each instance in SWE-bench Verified has been manually inspected to ensure task validity, making it a widely used standard for assessing autonomous coding agents. The benchmark comprises 500 instances drawn from widely-used Python projects on GitHub, where each instance presents a real bug report paired with a set of unit tests that determine whether a submitted patch correctly resolves the underlying issue. We additionally evaluate GPT-5-mini transfer performance on two external benchmarks: SWE-Rebench Leaderboard (110 instances across 41 repositories from the 2026.03 release) and SWE-QA, a repository-level code question answering benchmark. Together, they complement SWE-bench Verified by testing whether visual structural grounding generalizes to both new issue-resolution tasks and evidence-heavy repository QA scenarios.

To evaluate the performance and efficiency of the multimodal models under different design choices, we select the following metrics:

Pass@1: The percentage of issue-resolution benchmark instances for which the multimodal model generates a patch that successfully resolves the task. We report Pass@1 for SWE-bench Verified and SWE-Rebench.

Overall Score: The official aggregate score used by SWE-QA, reported on a 0–100 scale. This metric captures answer quality on repository-level code question answering tasks.

Number of API Calls: The average number of API calls made per benchmark instance, reflecting the number of interaction steps the agent takes to resolve an issue.

Number of Input Tokens: The average number of input tokens consumed per benchmark instance, capturing the context overhead introduced by the agent’s prompts and visual inputs.

Table 1: Effect of MLLMs on SWE-bench Verified (500 instances).

Agent	Effectiveness	Efficiency			
	Pass@1 ↑	API Calls ↓	Input Tokens ↓	Output Tokens ↓	Cost (\$) ↓
GPT-5-mini					
Text	55.0%	15	193,157	8,188	0.031
Vision-Only	41.4% (-13.6)	20 (+33%)	270,117 (+40%)	12,352 (+51%)	0.044 (+42%)
Doubao-Seed-2.0-Lite					
Text	51.0%	22	201,754	5,562	0.019
Vision-Only	16.9% (-34.1)	28 (+27%)	965,272 (+379%)	6,214 (+12%)	0.070 (+268%)
Kimi K2.5					
Text	70.3%	40	639,474	9,775	0.1213
Vision-Only	55.0% (-15.3)	78 (+95%)	879,853 (+38%)	9,258 (-5%)	0.1543 (+27%)

Number of Output Tokens: The average number of completion tokens generated per benchmark instance, reflecting the verbosity of the agent’s responses.

Average Cost per Instance: The average monetary cost incurred to process a single benchmark instance, computed by dividing the total evaluation cost by the number of evaluated instances.

Together, Pass@1 and Overall Score measure effectiveness for issue-resolution and QA settings respectively, while the remaining four metrics jointly characterize agent efficiency, enabling a comprehensive analysis of the performance-cost trade-off introduced by visual repository representations.

3.2 Implementation Details

Our framework is built on top of Mini-SWE-Agent and extends its tool-calling interface with a repository graph visualization module. For each target repository, we pre-construct a directed heterogeneous graph with four relation types—*contains*, *imports*, *invokes*, and *inherits*—and serialize it for reuse during inference. During inference, the agent queries the graph via an external tool, which renders the requested subgraph as a PNG image using Graphviz and returns it as visual context. All models are queried with temperature 0. Each agent run is limited to 250 interaction steps and a cost budget of \$3.0 per instance.

4 RQ1: Effectiveness of Current MLLMs at Issue Resolution tasks

We first investigate whether MLLMs can resolve repository-level issues. Specifically, we capture the outputs of the agent’s Bash navigation and file-reading commands to images and feed them to the MLLM.

4.1 Experimental Design

In the vision-only setting, the agent operates with its standard tool interface, but all Bash commands (e.g., `cat`, `find`, `grep`) return visual graph images rendered by SeeRepo instead of text output. The agent must navigate the repository, identify relevant code entities, and generate patches by interpreting these visual responses. We evaluate this setting on GPT-5-mini, Doubao-Seed-2.0-Lite, and Kimi K2.5 using the full SWE-bench Verified (500 instances).

4.2 Result Analysis

As shown in Table 1, vision-only context representation substantially reduces the accuracy for all three models. GPT-5-mini drops from 55.0% to 41.4% (−13.6), Doubao-Seed-2.0-Lite drops sharply from 51.0% to 16.9% (−34.1), and Kimi K2.5 drops from 70.3% to 55.0% (−15.3). All models also incur significantly higher costs: GPT-5-mini sees a 42% cost increase, Doubao 268%, and Kimi K2.5 27%. Notably, Kimi K2.5 nearly doubles its API calls (+95%), suggesting it compensates for the lack of textual information through more frequent but shorter queries.

When deprived of text representations, agents resort to repeated graph queries to compensate for missing precise symbolic information, accumulating high token overhead without improving accuracy. The three models exhibit distinct strategies under this constraint: Doubao-Seed-2.0-Lite engages most extensively with visual graphs, leading to a 379% input token surge and the steepest accuracy drop (−34.1); Kimi K2.5 adopts a high-frequency query strategy with 95% more API calls but only 38% more input tokens, maintaining relatively higher accuracy; while GPT-5-mini appears to abandon visual exploration earlier and generates patches with incomplete context. These results indicate that while MLLMs can process visual repository structure to varying degrees, graph images alone provide insufficient symbolic information for accurate issue resolution.

Finding

Vision-only repository interaction degrades issue resolution performance across all evaluated models: the resolution accuracy drops by 13.6 to 34.1 points, while token cost increases substantially (up to 268% higher cost). Agents resort to repeated graph queries to compensate for the absence of textual information, with API calls increasing by up to 95%.

5 RQ2: Effect of Multimodal Context Integration

Having seen that vision-only repository representation is ineffective, we investigate whether visual repository representation as a

Table 2: Performance comparison between text and multimodal agents (SeeRepo) across models.

Agent	Effectiveness	Efficiency			
	Pass@1 $\uparrow$	API Calls $\downarrow$	Input Tokens $\downarrow$	Output Tokens $\downarrow$	Cost (\$) $\downarrow$
GPT-5-mini					
Text	55.0%	15	193,157	8,188	0.031
Multimodal	55.4% (+0.4)	13 (-13%)	144,403 (-25%)	6,958 (-15%)	0.023 (-26%)
GPT-5.1					
Text	51.0%	16	206,256	3,113	0.1795
Multimodal	48.8% (-2.2)	13 (-19%)	161,130 (-22%)	2,342 (-25%)	0.0975 (-46%)
Kimi K2.5					
Text	68.8%	41	691,141	10,210	0.1270
Multimodal	70.6% (+1.8)	40 (-2%)	723,874 (+5%)	9,265 (-9%)	0.1229 (-3%)
Doubao-Seed-2.0-Lite					
Text	51.0%	22	185,311	5,383	0.0173
Multimodal	52.0% (+1.0)	21 (-4%)	176,463 (-4.8%)	4,958 (-7.9%)	0.0162 (-6.0%)

supplementary modality can improve issue resolution when integrated with standard text modality.

5.1 Methodology

To enable multimodal repository perception, we design SeeRepo, a tool that augments coding agents with visual renderings of repository structure. SeeRepo visually perceive the repository’s dependency graph alongside its standard text-based interface.

Initially, SeeRepo constructs a dependency graph for the repository via AST-based static analysis and renders query-centered subgraphs. Following LocAgent [6], we consider four types of directed relationships in the repository graph: *contains* (filesystem hierarchy), *imports* (module-level dependencies), *inherits* (class inheritance chains), and *invokes* (function-level call relationships). The graph is queriable at runtime by node identifier, edge type, and traversal depth in both upstream and downstream directions.

Next, SeeRepo transforms the dependency graph into structured visual representations. When the agent decides to query a node from the repository, it first constructs a bidirectional graph. This graph is built by performing breadth-first traversal over upstream and downstream relations up to a specified depth. The process constructs a distance-aware subgraph centered on the query target, where upstream dependencies are assigned negative distances and downstream dependencies are assigned positive distances. This arrangement captures both dependency flow and structural proximity, positioning each node according to its relative distance from the query target.

The constructed subgraph is then rendered using a layered, left-to-right hierarchical layout generated by the Graphviz DOT engine¹. Each node is displayed using a compact HTML-table label, augmented with semantic icons indicating entity types (e.g., files, modules, classes, or functions), while the queried node is visually highlighted to stabilize attention during reasoning. To reduce visual clutter in dense dependency regions, junction nodes are introduced to merge multiple outgoing edges before branching, improving

edge readability without altering graph semantics. This layered visualization not only makes dependency flow explicit but also reveals structural proximity by placing closely related entities at similar distances from the query target.

5.2 Experimental Setup

We evaluate SeeRepo on the bug localization task. The agent follows a structured three-phase localization strategy: (1) file hunting via the *imports* graph, (2) logic hunting via the *invokes* graph, and (3) hierarchy and path verification via *inherits* and *contains* as needed. Following the common localization pipeline, the agent reads relevant code snippets, implements a fix using standard Bash commands, and executes test cases to verify correctness. We evaluate SeeRepo on 500 instances from SWE-bench Verified using four models: GPT-5-mini, GPT-5.1, Kimi K2.5, and Doubao-Seed-2.0-Lite. To assess whether the same trend transfers beyond SWE-bench Verified, we additionally evaluate GPT-5-mini on SWE-Rebench Leaderboard and SWE-QA. For SWE-Rebench Leaderboard, we use all 110 instances from the 2026.03 release, spanning 41 repositories. For SWE-QA, we follow the official evaluation protocol and report the Overall Score (0–100) together with the same efficiency statistics used in our main experiments.

5.3 Results and Analysis

As shown in Table 2, integrating multimodal context substantially reduces token consumption while improving or preserving accuracy: GPT-5-mini reduces total cost by 26%, GPT-5.1 by 46%, and Doubao-Seed-2.0-Lite by 6%. This consistent reduction suggests that structural context primarily improves the *localization phase* of issue resolution. In text-only interaction, agents typically rely on iterative file exploration, repeatedly issuing navigation commands to refine hypotheses about relevant code regions. By contrast, explicit structural grounding enables agents to narrow the candidate search space earlier, reducing redundant exploration and shortening reasoning trajectories. Concretely, a single graph query exposes the full dependency neighborhood of a target node in one step,

¹<https://graphviz.org/>

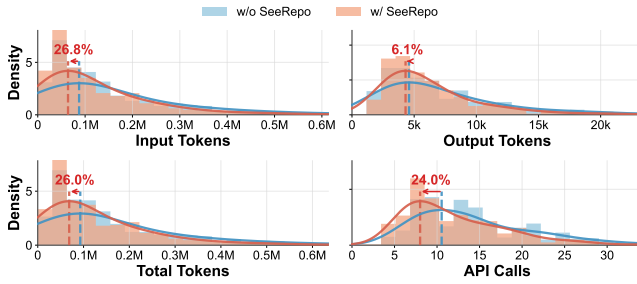


Figure 3: Efficiency analysis on SWE-bench Verified. *w/ SeeRepo* achieves substantial reductions in both token cost and agent rounds compared to *w/o SeeRepo* for GPT-5-mini.

short-circuiting the iterative grep-then-read cycle that otherwise requires multiple sequential navigation commands to accumulate equivalent structural context.

The magnitude of efficiency gains varies across models, revealing differences in how models internalize structural signals. GPT-5.1 exhibits the largest cost reduction (−46%) despite a slight accuracy drop (−2.2), suggesting that stronger base reasoning models already possess effective repair capabilities but benefit from structural context as a navigation prior that accelerates repository understanding. The accuracy regression likely reflects a tension specific to high-capability models: GPT-5.1’s stronger parametric reasoning already allows it to form reliable localization hypotheses from sparse textual cues, so the additional graph queries occasionally redirect attention toward dependencies that are topologically proximate but semantically tangential to the defect. In this case, structural grounding trades a small amount of repair precision for substantially improved exploration efficiency.

Kimi K2.5 presents a contrasting pattern: input tokens increase slightly (+5%), yet accuracy improves and overall cost still decreases by 3%. Rather than using graph queries to replace textual exploration, Kimi K2.5 appears to treat them as supplementary context that reinforces its ongoing hypothesis formation: the near-unchanged API call count (41→40) alongside a 5% input token increase suggests that structural and symbolic signals are processed in tandem rather than as substitutes. This more deliberative integration results in modestly longer inputs but more reliable downstream repair decisions. The outcome—highest Pass@1 (70.6%) among all configurations—suggests that broadening context acquisition under coherent structural guidance can improve repair robustness even at the cost of some exploration overhead.

Doubao-Seed-2.0-Lite shows moderate accuracy gains with smaller efficiency improvements, suggesting that multimodal context primarily streamlines repository navigation without substantially changing the overall interaction pattern. Compared with other models, the reduction in token usage is more limited, indicating that visual representations mainly help agents reduce redundant exploration steps rather than reshaping the repair process.

Figure 3 further illustrates efficiency differences on GPT-5-mini by comparing the text-only baseline with the multimodal context augmented with SeeRepo. The multimodal setting exhibits a clear leftward shift in the distribution of input tokens, indicating that the visual representation of repository structure reduces the amount of

Table 3: Additional GPT-5-mini evaluation on SWE-Rebench Leaderboard. The effectiveness metric is Pass@1 over the 110-instance 2026.03 release.

Agent	Effectiveness	Efficiency			
	Pass@1	API Calls	Input Tokens	Output Tokens	Cost (\$)
Text	25.45%	23	388,834	12,094	0.052
Multimodal	26.36% (+0.91)	21 (-2)	253,132 (-34.89%)	10,242 (-15.32%)	0.047 (-9.6%)

Table 4: Additional GPT-5-mini evaluation on SWE-QA. The effectiveness metric is the official Overall Score (0–100).

Agent	Effectiveness	Efficiency			
	Score	API Calls	Input Tokens	Output Tokens	Cost (\$)
Text	66.8	14	66,369	5,003	0.0107
Multimodal	67.2 (+0.4)	9 (-35.7%)	50,038 (-24.6%)	4,254 (-15.0%)	0.0079 (-26.2%)

repository context the agent needs to inspect before identifying relevant files. Reductions in output tokens are smaller but consistent, suggesting that SeeRepo primarily improves upstream exploration efficiency rather than substantially altering the verbosity of patch generation once a repair direction is established. This trend is reflected in total token usage, where the overall distribution shifts left by 26.0%, confirming that efficiency gains are driven mainly by reduced context acquisition rather than shorter responses alone. Additionally, the multimodal configuration requires fewer API calls, indicating that visual modality enables the agent to localize relevant code regions with fewer iterative navigation and verification steps compared to the text-only baseline.

Taken together, these results suggest that multimodal context integration improves issue resolution not by replacing textual reasoning, but by guiding repository exploration. The structural visualization provides a global view of repository structure, allowing agents to identify relevant regions earlier and avoid redundant navigation steps. As a result, reasoning trajectories become shorter and more focused, reducing token consumption while maintaining repair accuracy across different models. The magnitude of token reduction varies across models, reflecting differences in exploration and tool-usage behaviors.

As shown in Tables 3 and 4, the multimodal setting preserves or slightly improves effectiveness on both benchmarks while reducing interaction cost. On SWE-Rebench Leaderboard, SeeRepo raises Pass@1 from 25.45% to 26.36% while cutting input tokens by 34.89% and cost by 9.6%. On SWE-QA, it improves the Overall Score from 66.8 to 67.2 while reducing API calls by 35.7% and cost by 26.2%. The larger efficiency gain on SWE-QA is consistent with the role of structural visualization in localization-heavy tasks: question answering mainly requires identifying the relevant code region rather than completing the full repair pipeline.

Finding

Multimodal context integration reduces token cost by up to 46% on SWE-bench Verified while maintaining or improving accuracy, and the same effectiveness-efficiency trend transfers to additional GPT-5-mini evaluations on SWE-Rebench and SWE-QA.

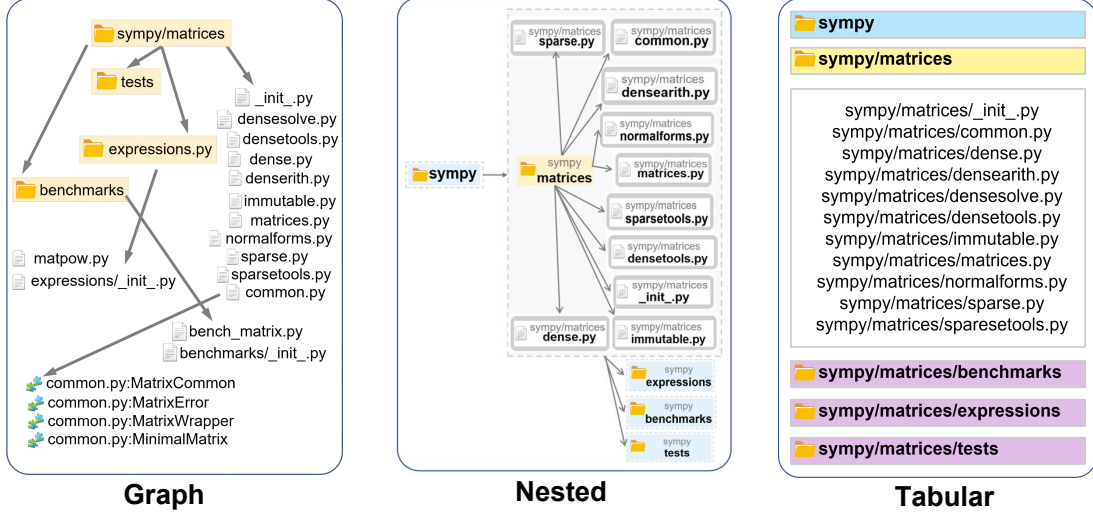


Figure 4: Examples of Three Visualization Styles.

6 RQ3: Effect of Visual Layout

With the efficiency of multimodal integration verified, we further investigate which visual layout is the most effective for coding agents. We compare three visual rendering of repository, including graph, nested, and tabular. Additionally, we examine the effect of hierarchy depth within the visual representations.

6.1 Experimental Design

We compare three visual rendering strategies, as illustrated in Figure 4. The three variants render the repository subgraph as images. **Graph** renders the subgraph as a directed graph where nodes are connected by edges. Node types are distinguished using different icons (e.g., folder icons for directories, document icons for files), while dependency directions are encoded through arrow orientation, preserving the full topological structure of the dependency graph. **Nested** extends the graph layout by grouping nodes belonging to the same directory or file within dashed bounding boxes, making hierarchical containment spatially explicit without requiring the agent to trace *contains* edges. **Tabular** removes explicit edges entirely and presents nodes as a flat, color-coded list: the query node in yellow, its parent directory in blue, subdirectories in purple, and contained files grouped in white blocks, encoding relational context through color rather than topology. In addition to the visual layouts, we consider a textual rendering strategy as a reference. **Text** linearizes the repository structure queried by SeeRepo into a sequential textual format (e.g., listing nodes and their relationships as structured text).

Hierarchy Depth. When the agent queries the graph tool, it specifies a query node and a traversal depth k . The graph tool returns all nodes reachable within k hops along the specified edge type. A larger k exposes a broader dependency neighborhood but also increases the size of the rendered image and the resulting input token count. To study this tradeoff, we fix the traversal depth at $k \in \{1, 2, 3, 4\}$ and compare these settings with SeeRepo, where the

Table 5: Comparison of visualization layout on SWE-bench Verified (GPT-5 mini, 500 instances).

Method	Pass@1	Input Tokens	Output Tokens	Cost (\$)
Mini-SWE-Agent	55.0%	193,157	8,188	0.031
+ Text	53.8%	159,558 (−17%)	8,380 (+2%)	0.027 (−12%)
+ Graph	55.4%	144,403 (−25%)	6,958 (−15%)	0.023 (−26%)
+ Nested	55.8%	154,788 (−20%)	7,458 (−9%)	0.025 (−18%)
+ Tabular	56.2%	163,311 (−16%)	7,545 (−8%)	0.027 (−14%)

agent dynamically determines k for each query. All experiments use GPT-5-mini on 500 instances from SWE-bench Verified. The Graph layout with agent-decided depth corresponds to the default SeeRepo configuration evaluated in RQ2.

6.2 Results and Analysis

As shown in Table 5, all three visual layouts improve over the text-only baseline. The text representation reduces input tokens by 17% but marginally hurts accuracy (−1.2), indicating that structured text alone cannot fully substitute for visual structural context. Among visual layouts, graph layout achieves the best token efficiency (−25% input tokens, −26% cost) at a modest accuracy gain (+0.4), while tabular layout yields the highest Pass@1 (56.2%, +1.2) at a lower efficiency gain (−16% cost). Nested layout sits between the two (Pass@1 55.8%, −18% cost). This tradeoff suggests that graph layout encodes structural relationships more compactly for token-efficient navigation, whereas tabular layout with semantic color-coding provides richer local context that aids precise localization.

As shown in Table 6, fixed hop depths all reduce cost relative to the baseline. Depth 1 slightly hurts accuracy (−0.6) as the shallow neighborhood may omit key dependencies; deeper depths progressively improve accuracy, with Depth 4 achieving the highest Pass@1 (57.2%, +2.2). However, fixed depths incur more input tokens as depth increases. SeeRepo with agent-decided depth achieves a competitive Pass@1 gain (+0.4) with the lowest input tokens (−25%) and

Table 6: Effect of hierarchy depth on SWE-bench Verified (GPT-5-mini, 500 instances). SeeRepo uses an adaptive depth determined by agents.

Graph Depth	Pass@1	Input Tokens	Output Tokens	Cost (\$)
Mini-SWE-Agent	55.0%	193,157	8,188	0.031
Depth = 1	54.4% (-0.6)	155,971 (-19%)	9,577 (+17%)	0.028 (-10%)
Depth = 2	55.8% (+0.8)	158,350 (-18%)	7,782 (-5%)	0.026 (-15%)
Depth = 3	55.4% (+0.4)	156,019 (-19%)	7,624 (-7%)	0.026 (-16%)
Depth = 4	57.2% (+2.2)	161,441 (-16%)	7,466 (-9%)	0.026 (-16%)
SeeRepo	55.4% (+0.4)	144,403 (-25%)	6,958 (-15%)	0.023 (-26%)

cost (-26%) across all depth configurations, suggesting the agent adaptively selects shallow depths when a narrow context suffices and deeper traversals only when required.

Finding

All visual layouts outperform the text-only baseline, with graph layout achieving the best token efficiency (-26% cost). Agent-decided hop depth achieves a competitive Pass@1 gain (+0.4) while delivering the largest reductions in input tokens (-25%) and cost (-26%) across all depth configurations.

7 RQ4: Effectiveness of Visualization in Different Stages

With multimodal integration shown to improve both effectiveness and efficiency, we further investigate *when* visualization is most beneficial within the issue resolution pipeline. The contribution of structural visual context may vary across different stages of problem solving, as early stages of issue resolution primarily involve bug localization, whereas later stages focus on code modification and validation.

7.1 Experimental Design

Issue resolution proceeds through three distinct stages: bug localization, patch repair, and patch validation [34, 35]. To isolate the effect of invocation stage, we construct three variants that each confine visualization-tool invocation to a single stage. The localization variant corresponds to the default SeeRepo setting; the other two are ablations that shift invocation to later stages:

Localization Variant. To isolate the effect of structural grounding during early issue analysis, the agent is equipped with the SeeRepo graph tool only in the localization phase, where it explores repository structure and identifies relevant code entities. All subsequent stages rely solely on standard Bash commands.

Repair Variant. To evaluate the role of structural information during code modification, the agent uses standard Bash commands for navigation and editing, but invokes the SeeRepo graph tool before applying changes. The tool is used to inspect upstream and downstream dependencies of the target entity, helping avoid unintended side effects during patch construction.

Patch Validation Variant. To examine whether structural grounding primarily benefits post-modification verification, the agent performs localization and repair using standard Bash commands only. After generating a patch, the agent invokes the SeeRepo graph tool

to inspect the dependency neighborhood of the modified entity before generating and executing validation tests.

Table 7: Effect of visualization in different stages on SWE-bench Verified (GPT-5-mini, 500 instances). The percentages in parentheses indicate relative change to Mini-SWE-Agent.

Stage	Pass@1	Input Tokens	Output Tokens	Cost (\$)
Mini-SWE-Agent	55.0%	193,157	8,188	0.031
Localization	55.4% (+0.4)	144,403 (-25%)	6,958 (-15%)	0.023 (-26%)
Repair	50.0% (-5.0)	174,544 (-10%)	8,879 (+8%)	0.029 (-5%)
Patch Validation	51.6% (-3.4)	178,922 (-7%)	8,758 (+7%)	0.030 (-4%)

7.2 Results and Analysis

As shown in Table 7, the effectiveness of visualization varies substantially depending on the stage at which it is invoked. Enabling visualization during the repair stage yields a Pass@1 of 50.0% (-5.0) while providing only a marginal 5% reduction in cost. After localization has already identified candidate files, invoking the graph tool during repair exposes the agent to a broader set of upstream and downstream dependencies, many of which are not directly relevant to the target fix. This additional structural context appears to introduce distraction, interfering with the precise textual reasoning required for accurate code editing.

Deferring visualization to the patch validation stage partially recovers performance (51.6%, -3.4) but still underperforms the baseline. At this point, structural information is introduced only after a patch has been produced; inspecting dependency neighborhoods may surface seemingly related components and encourage unnecessary follow-up modifications, potentially expanding patch scope and increasing the likelihood of regressions.

In contrast, introducing SeeRepo to the localization stage produces the best overall results, achieving a Pass@1 of 55.4% (+0.4) together with a 25% reduction in input tokens and a 26% decrease in cost. During early bug localization, visual modality helps the agent narrow the candidate search space and identify relevant code entities prior to repair, reducing redundant exploration while preserving accurate trajectory reasoning.

Finding

Visualization is most effective at the fault localization stage: SeeRepo achieves Pass@1 of 55.4% (+0.4) with 26% cost reduction. Invoking it at repair or validation stages degrades accuracy by up to 5.0 points, as late-stage structural context introduces noise rather than useful signal.

8 Case Study

To qualitatively illustrate how SeeRepo reduces exploratory overhead, we showcase the instance `astropy__astropy-13398` from SWE-bench Verified, which requires implementing `direct ITRS↔AltAz` and `ITRS↔HADec` coordinate transformations in the `astropy` library by creating `itr_observed_transforms.py` and registering it in `builtin_frames/__init__.py`. Figure 5 contrasts the localization trajectories of the baseline agent and SeeRepo on this instance.

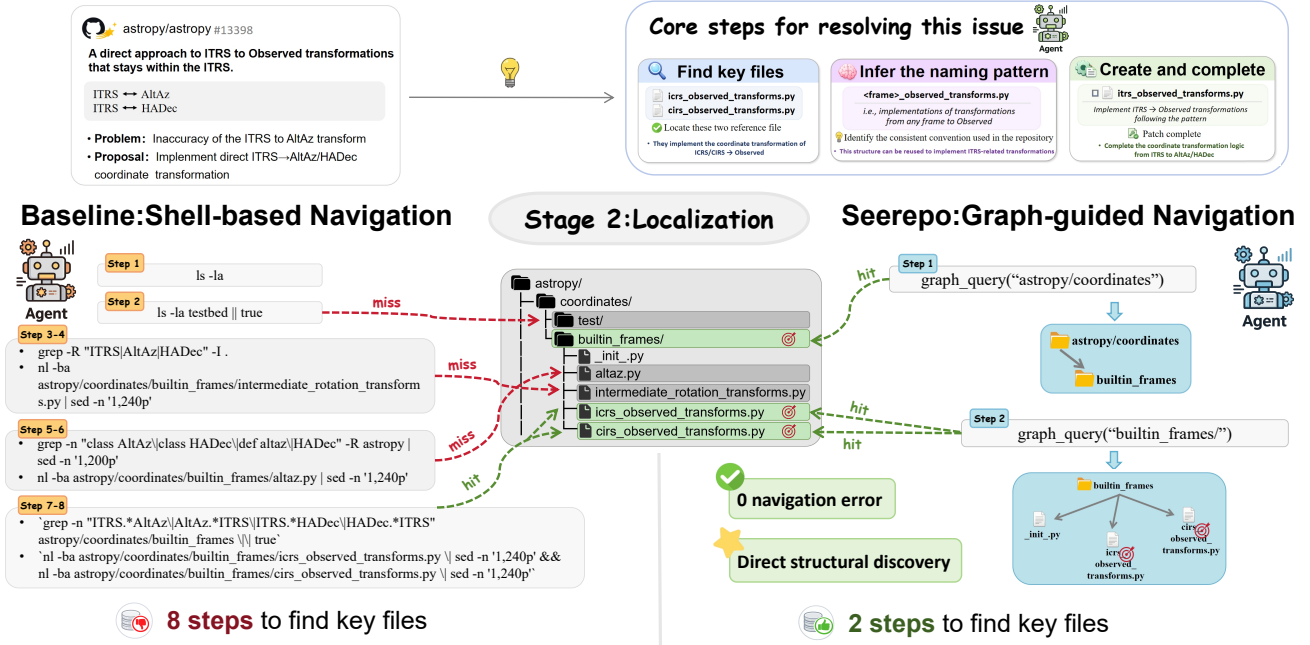


Figure 5: Case study on SWE-bench Verified instance `astropy__astropy-13398`. The baseline agent relies on iterative shell-based exploration to locate relevant files, incurring repeated misses and redundant context consumption. In contrast, SeeRepo integrates multimodal repository information by combining structured graph-based repository context with textual code access, enabling direct structural discovery, fewer navigation errors, and more efficient localization.

Without structural context, the baseline agent resorts to an iterative `grep`-then-read strategy. After initial repository inspection via `ls -la` (Steps 1–2), it launches a broad keyword search (`grep -R "ITRS|AltAz|HADec"`) across the entire repository, which returns voluminous but low-relevance output (Step 3, *miss*). The agent then reads `intermediate_rotation_transforms.py` to infer the frame structure (Step 4, *miss*), and further issues targeted class-level searches (`grep -n "class AltAz|class HADec..."`) followed by reading `altaz.py` (Steps 5–6, *miss*). Only at Steps 7–8, after additional pattern-matching queries, does the agent finally locate `icrs_observed_transforms.py` and `cirs_observed_transforms.py`—the template files that reveal the naming convention and registration mechanism needed for the fix. In total, eight interaction steps are consumed before the key files are identified, with three rounds of *miss* preceding the eventual *hit*.

In contrast, the multimodal agent equipped with SeeRepo enables the agent to reach the same structural understanding in two steps. The agent first issues `graph_query("astropy/coordinates")` (Step 1), which returns the contains-edge subgraph rooted at this directory and immediately surfaces `builtin_frames/` as a relevant subdirectory (*hit*). A follow-up query `graph_query("builtin_frames/")` (Step 2) retrieves the complete file roster of that directory (*hit*), directly exposing the naming pattern `*_observed_transforms.py` instantiated by `cirs_observed_transforms.py`

and `icrs_observed_transforms.py`. This structured response provides both an implementation template and implicit guidance on the registration mechanism, allowing the agent to proceed to creating `itrs_observed_transforms.py` with zero navigation errors.

While both agents ultimately produce correct patches, SeeRepo reduces total token consumption by 32.6% (143,558 → 96,816) and interaction steps by 29% (17 → 12). The efficiency gap originates in the localization stage: the baseline’s repeated `grep` outputs and file reads accumulate approximately 25K tokens of low-information-density context before productive modification begins, whereas two graph queries provide equivalent—and more structured—information at a fraction of the token cost.

This case illustrates the core benefit of SeeRepo: by replacing trial-and-error shell exploration with topology-aware queries, agents can devote more of their limited context window to reasoning and code modification rather than navigation.

9 Discussion

9.1 Threats to Validity

External Validity. Our evaluation is conducted on SWE-bench Verified, which consists exclusively of Python repositories. Although this benchmark provides realistic issue-resolution scenarios with reliable test-based validation, the observed benefits of structural visualization may depend on characteristics specific to Python projects,

such as module organization patterns and dependency structures. Consequently, it remains unclear whether the same efficiency and reasoning improvements would generalize to repositories written in other programming languages (e.g., Java or TypeScript) that exhibit different architectural conventions or build systems.

Construct Validity. We measure efficiency primarily through token consumption and reasoning trajectory length, using them as proxies for exploration efficiency. While these metrics capture computational cost and navigation behavior, they may not fully reflect qualitative aspects of reasoning, such as interpretability or developer-aligned debugging strategies. Future studies incorporating human evaluation or finer-grained behavioral analyses could provide a more comprehensive assessment of agent reasoning quality.

Internal Validity. To isolate the effect of structural visualization, all experimental settings are kept identical to the text-only baseline except for the addition of structural context. Nevertheless, interactions between visualization and specific model architectures may still influence outcomes. For example, models with different planning or tool-usage tendencies may exploit structural context to varying degrees, potentially affecting token reduction ratios across models.

9.2 Future Work

Several promising directions remain for future exploration. First, the current structural visualization relies on static Graphviz layouts, which may become visually dense when applied to large-scale repositories with complex dependency structures. Developing adaptive visualization strategies that dynamically emphasize query-relevant subgraphs or progressively reveal structural information could substantially improve interpretability and scalability.

Second, while our current framework allows agents to decide the depth of structural exploration, more principled mechanisms for controlling visualization scope remain unexplored. Learning-based invocation policies, such as reinforcement learning or uncertainty-aware triggering mechanisms, could enable agents to request structural context only when it is expected to provide measurable reasoning benefits.

Finally, extending structural grounding beyond static analysis is an important avenue for future work. Incorporating dynamic signals, such as execution traces or runtime dependencies, may enable richer representations of repository behavior and further enhance agent reasoning in complex software environments. Such hybrid representations could also help agents distinguish between frequently executed code paths and rarely triggered branches, enabling more targeted localization and repair.

10 Related Work

Software Engineering Agents. Recent years have seen rapid progress on LLM-based agents [4, 14, 22–25, 27, 31, 32, 39] for repository-level issue resolution. On the scaffold side, SWE-agent [35] demonstrates that an agent-computer interface (ACI) that supports repository navigation, editing, and execution can substantially improve task performance. AutoCodeRover [40] further incorporates software engineering oriented context retrieval, leveraging

an AST-based program representation (e.g., class/method structure) and iterative search to ground patch generation for GitHub issues. Recent work also improves repository navigation and localization using structured signals and scheduling: LocAgent [6] uses graph-guided multi-hop traversal to localize relevant entities, RepoMem [30] augments localization with repository memory mined from history, and OrcaLoca [38] improves localization with scheduling and distance-aware context pruning. Building on such scaffolds, experience-driven approaches aim to reuse past repair knowledge rather than treating each issue in isolation: SWE-Exp [5] constructs an experience bank from historical trajectories to guide planning and patching, while ExpeRepair [18] introduces a dual-memory design (episodic demonstrations and semantic reflections) to dynamically compose prompts for repository-level repair. Orthogonally, test-time scaling and search-based methods increase inference-time compute to explore and refine candidate solutions: SWE-Debate [13] uses competitive multi-agent debate (and integrates search during patching) to improve fault localization and fix planning, and SWE-Search [1] augments agents with Monte Carlo Tree Search and iterative refinement to enable backtracking and deeper exploration. Relatedly, SAGE [9] improves agent behavior by self-abstracting grounded experience into compact plans for subsequent re-execution, providing a complementary path to boost long-horizon performance at test time. In contrast to improving agents via new scaffolds, memories, or inference-time search, our work is the first to study multimodal representations of code repositories as a design dimension for SWE agents. We find that representing repository structure as visual graph images consistently reduces token cost while maintaining resolution accuracy, and that the benefit is most pronounced when visualization is invoked at the localization stage.

Multimodal Coding Agents. As more real-world software issues are reported with visual evidence (e.g., screenshots), recent benchmarks have begun to evaluate agents in software engineering with visual element information. SWE-bench Multimodal (SWE-bench M) [36] extends SWE-bench to visual, user-facing JavaScript repositories by providing issue resolution tasks that include images in problem statements or tests, enabling evaluation in visual software domains. Building on this benchmark, recent work has started to incorporate multimodal signals into coding agents and automated repair. GUIRepair [10] studies visual software issue fixing by enabling cross-modal reasoning between GUI screenshots and code, and by using rendered visual feedback to support patch validation. OpenHands-Versa [28] augments coding agents with multimodal browsing as a generalist capability and evaluates on benchmarks including SWE-bench Multimodal [36]. SVRepair [29] proposes structured visual reasoning for automated program repair by transforming heterogeneous visual artifacts into semantic scene graphs to guide localization and patch synthesis. Beyond incorporating external visual artifacts, recent work has also explored representing code itself through visual modalities. CodeOCR [26] renders source code as images to enable multimodal models to process programs with improved token efficiency while maintaining performance on code understanding tasks. This line of work suggests that visual representations can serve as an alternative interface for presenting software information to foundation models. In contrast, SeeRepo adopts a multimodal design that visualizes repository structure

rather than code content. Instead of encoding individual files as images, SeeRepo constructs structural visualizations derived from static analysis, exposing dependency relationships and global repository organization that are often implicit in text-only interaction. In this formulation, repository structure is presented through a visual modality, while fine-grained code details remain in their original textual form for precise semantic reasoning. Our work therefore examines how structural visual context affects issue resolution in realistic software engineering (SWE) tasks, providing a complementary perspective on multimodal representations beyond code-centric visual encoding.

11 Conclusion

This paper presents the first systematic empirical study of visual repository representations for MLLM-based software engineering agents on repository-level issue resolution. We introduce SeeRepo, a multimodal framework that presents different types of repository information through appropriate modalities: structural and dependency relationships are rendered as visual graph images, while code content is retained as text. This design leverages the complementary strengths of MLLMs—visual perception for structural orientation and symbolic reasoning for precise code understanding—enabling agents to navigate large repositories more effectively.

Our experiments on SWE-bench Verified with four models yield four key findings. First, vision-only modality is insufficient: replacing text access with graph images degrades accuracy by up to 34.1 points while paradoxically inflating token cost. Second, multimodal integration—adding SeeRepo alongside standard text tools—reduces cost by up to 46% across all models while maintaining or improving resolution accuracy. Third, among visual layout designs, graph layout provides the best token efficiency and agent-decided hop depth achieves the best cost reduction while maintaining competitive accuracy. Fourth, visualization is most effective at the localization stage; invoking it during repair or validation degrades performance due to noise introduction and scope expansion.

These findings collectively suggest that visual repository representations are a practical and cost-effective complement to text-based interaction for coding agents, provided they are designed and invoked appropriately. More broadly, our results indicate that the modality through which structural information is presented—not merely its content—meaningfully shapes agent behavior and cost. We hope this work motivates further exploration of multimodal representations in the development of future coding agents.

References

- [1] Antonis Antoniadis, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. 2024. SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement. *arXiv:2410.20285*
- [2] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2026. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *Advances in Neural Information Processing Systems* 38 (2026).
- [3] ByteDance. 2026. Doubao Seed 2.0: General-Purpose Agent Models. <https://www.volcengine.com>. Released February 2026.
- [4] Pengyu Chang, Yixiong Fang, Silin Chen, Yuling Shi, Beijun Shen, and Xiaodong Gu. 2026. Test vs Mutant: Adversarial LLM Agents for Robust Unit Test Generation. *arXiv preprint arXiv:2602.08146* (2026).
- [5] Silin Chen, Shaoxin Lin, Xiaodong Gu, Yuling Shi, Heng Lian, Longfei Yun, Dong Chen, Weiguo Sun, Lin Cao, and Qianxiang Wang. 2025. Swe-exp: Experience-driven software issue resolution. *arXiv preprint arXiv:2507.23361* (2025).
- [6] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. *arXiv preprint arXiv:2503.09089* (2025).
- [7] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [9] Hiroaki Hayashi, Bo Pang, Wenting Zhao, Ye Liu, Akash Gokul, Srijan Bansal, Caiming Xiong, Semih Yavuz, and Yingbo Zhou. 2025. Self-Abstraction from Grounded Experience for Plan-Guided Policy Refinement. *arXiv preprint arXiv:2511.05931* (2025).
- [10] Kai Huang, Jian Zhang, Xiaofei Xie, and Chunyang Chen. 2025. Seeing is Fixing: Cross-Modal Reasoning with Multimodal LLMs for Visual Software Issue Fixing. *arXiv:2506.16136 [cs.SE]* <https://arxiv.org/abs/2506.16136>.
- [11] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. Issue Localization via LLM-Driven Iterative Code Graph Searching. *arXiv preprint arXiv:2503.22424* (2025).
- [12] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).
- [13] Han Li, Yuling Shi, Shaoxin Lin, Xiaodong Gu, Heng Lian, Xin Wang, Yantao Jia, Tao Huang, and Qianxiang Wang. 2025. Swe-debate: Competitive multi-agent debate for software issue resolution. *arXiv preprint arXiv:2507.23348* (2025).
- [14] Yalan Lin, Yingwei Ma, Rongyu Cao, Binhua Li, Fei Huang, Xiaodong Gu, and Yongbin Li. 2024. Llms as continuous learners: Improving the reproduction of defective code in software issues. *arXiv preprint arXiv:2411.13941* (2024).
- [15] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. 2024. GraphCoder: Enhancing Repository-Level Code Completion via Code Context Graph-based Retrieval and Language Model. *arXiv preprint arXiv:2406.07003* (2024).
- [16] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Shieh, and Wenmeng Zhou. 2024. CodexGraph: Bridging Large Language Models and Code Repositories via Code Graph Databases. *arXiv preprint arXiv:2408.03910* (2024).
- [17] Moonshot AI. 2026. Kimi K2.5: Native Multimodal Agentic Model. <https://kimi.moonshot.cn>. Released January 2026.
- [18] Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. 2025. Experepair: Dual-memory enhanced llm-based repository-level program repair. *arXiv preprint arXiv:2506.10484* (2025).
- [19] OpenAI. 2024. SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>.
- [20] OpenAI. 2025. GPT-5: OpenAI’s Next Generation Language Model. <https://openai.com>. Released August 2025.
- [21] OpenAI. 2025. GPT-5.1: Enhanced Reasoning and Personalization. <https://openai.com>. Released November 2025.
- [22] Weihai Peng, Yuling Shi, YINGWEI MA, Longfei Yun, Beijun Shen, and Xiaodong Gu. [n. d.]. DeepRepoQA: Code Repository Question Answering with Deep Agent Exploration. ([n. d.]).
- [23] Weihai Peng, Yuling Shi, Yuhang Wang, Xinyun Zhang, Beijun Shen, and Xiaodong Gu. 2025. Swe-qa: Can language models answer repository-level code questions? *arXiv preprint arXiv:2509.14635* (2025).
- [24] Yuling Shi, Yichun Qian, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2025. Longcodezip: Compress long context for code language models. *arXiv preprint arXiv:2510.00446* (2025).
- [25] Yuling Shi, Songsong Wang, Chengcheng Wan, Min Wang, and Xiaodong Gu. 2024. From code to correctness: Closing the last mile of code generation with hierarchical debugging. *arXiv preprint arXiv:2410.01215* (2024).
- [26] Yuling Shi, Chaoxiang Xie, Zhensu Sun, Yeheng Chen, Chenxu Zhang, Longfei Yun, Chengcheng Wan, Hongyu Zhang, David Lo, and Xiaodong Gu. 2026. CodeOCR: On the Effectiveness of Vision Language Models in Code Understanding. *arXiv preprint arXiv:2602.01785* (2026).
- [27] Yuling Shi, Hongyu Zhang, Chengcheng Wan, and Xiaodong Gu. 2025. Between lines of code: Unraveling the distinct patterns of machine and human programmers. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1628–1639.
- [28] Aditya Bharat Soni, Boxuan Li, Xingyao Wang, Valerie Chen, and Graham Neubig. 2025. Coding Agents with Multimodal Browsing are Generalist Problem Solvers. *arXiv:2506.03011 [cs.CL]* <https://arxiv.org/abs/2506.03011>.
- [29] Xiaoxuan Tang, Jincheng Wang, Liwei Luo, Jingxuan Xu, Sheng Zhou, Dajun Chen, Wei Jiang, and Yong Li. 2026. SVRepair: Structured Visual Reasoning for Automated Program Repair. *arXiv:2602.06090 [cs.SE]* <https://arxiv.org/abs/2602.06090>.

- [30] Boshi Wang, Weijian Xu, Yunsheng Li, Mei Gao, Yujia Xie, Huan Sun, and Dongdong Chen. 2025. Improving Code Localization with Repository Memory. *arXiv preprint arXiv:2510.01003* (2025).
- [31] Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026. SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents. *arXiv preprint arXiv:2601.16746* (2026).
- [32] Yifei Wang, Ziteng Wang, Yuling Shi, Silin Chen, Xinrui Wang, Yueqi Wang, Beijun Shen, Linjing Li, Xiaodong Gu, Julian McAuley, et al. 2026. Context Compression for LLM Agents: A Survey of Methods, Failure Modes, and Evaluation. (2026).
- [33] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [34] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489 [cs.SE] <https://arxiv.org/abs/2407.01489>
- [35] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [36] John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, Diyi Yang, Sida I Wang, and Ofir Press. 2025. SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains?. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=riTiQ3i21b>
- [37] Jiabo Ye, Anwen Hu, Haiyang Xu, Qinghao Ye, Ming Yan, Yuhao Dan, Chenlin Zhao, Guohai Xu, Chenliang Li, Junfeng Tian, et al. 2023. mPLUG-DocOwl: Modularized Multimodal Large Language Model for Document Understanding. *arXiv preprint arXiv:2307.02499* (2023).
- [38] Zhongming Yu, Hejia Zhang, Yujie Zhao, Hanxian Huang, Matrix Yao, Ke Ding, and Jishen Zhao. 2025. OrcaLoca: An LLM Agent Framework for Software Issue Localization. *arXiv preprint arXiv:2502.00350* (2025).
- [39] Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, et al. 2026. SWE-Explore: Benchmarking How Coding Agents Explore Repositories. *arXiv preprint arXiv:2606.07297* (2026).
- [40] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. arXiv:2404.05427 [cs.SE] <https://arxiv.org/abs/2404.05427>.